

Όραση Υπολογιστών

1η Σειρά Εργαστηριακών Ασκήσεων

Ραφτόπουλος Μιχαήλ - ΑΜ: 03120114

Εαρινό Εξάμηνο 2024

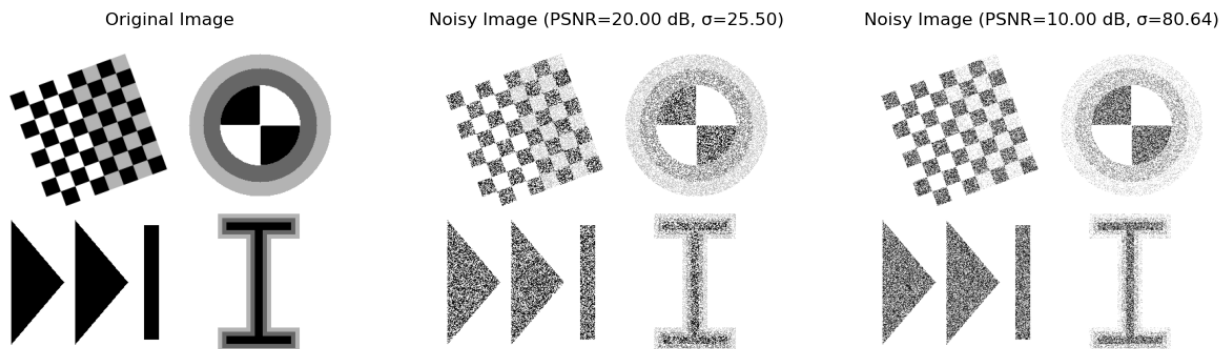
Σημείωση: Προκειμένου να διατηρηθεί συνοπτική η αναφορά, αφαιρέθηκε ο πηγαίος κώδικας. Εάν θέλετε να δείτε την αναφορά με τον πηγαίο κώδικα, κοιτάξτε το αρχείο 'lab1_report_withcode.pdf'.

Μέρος 1: Ανίχνευση Ακμών σε Γκρίζες Εικόνες

1.1 Δημιουργία Εικόνων Εισόδου

1.1.1 Διαβάζουμε την εικόνα εισόδου και την μετατρέπουμε από BGR σε Grayscale.

1.1.2 Προκειμένου να βρεθούν οι κατάλληλες τυπικές αποκλίσεις σ_n με ακρίβεια ώστε να έχουμε τα ζητούμενα P-SNR, αλλά και για ευκολότερη μελλοντική χρήση, υλοποιήθηκε η συνάρτηση PSNRnoise(). Η συνάρτηση αυτή δέχεται μια εικόνα και μια τιμή PSNR και της εφαρμόζει λευκό γκαουσιανό θόρυβο, με την κατάλληλη τυπική απόκλιση σ_n . Για τον υπολογισμό αυτό χρησιμοποιείται ένας απλός αλγόριθμος δυαδικής αναζήτησης. Εφαρμόζουμε τον θόρυβο στην εικόνα και παρουσιάζουμε τα αποτελέσματα

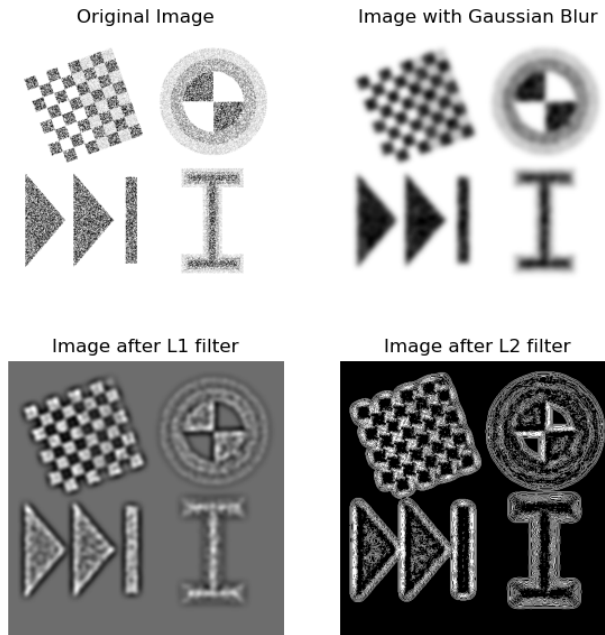


Σχήμα 1: Η αρχική εικόνα, εικόνα με γκαουσιανό θόρυβο PSNR=20dB, εικόνα με γκαουσιανό θόρυβο PSNR=10dB

1.2 Υλοποίηση Αλγορίθμων Ανίχνευσης Ακμών

1.2.1 Υλοποιήθηκαν οι δύο συναρτήσεις που επιστρέφουν τις ζητούμενες αποκρίσεις. Και οι δύο συναρτήσεις χρησιμοποιούν αυθαίρετα $\sigma = 5$. Το βέλτιστο σ θα βρεθεί αργότερα στην εργασία.

1.2.2 Εφαρμόζουμε τα ζητούμενα φίλτρα L_1 και L_2 και εμφανίζουμε τα αποτελέσματα. Στην προκειμένη περίπτωση χρησιμοποιείται η θορυβοποιημένη εικόνα με PSNR=20dB.



Σχήμα 2: Η αρχική εικόνα, μετά από το φίλτρο G, μετά το φίλτρο LoG, μετά τα μορφολογικά φίλτρα

1.2.3 Γίνεται η προσέγγιση των zero-crossings. Χρησιμοποιείται ο προτεινόμενος αλγόριθμος, δημιουργείται δηλαδή η δυαδική εικόνα και εντοπίζεται το περίγραμμα με χρήση των κατάλληλων μορφολογικών τελεστών. Εμφανίζονται τα αποτελέσματα



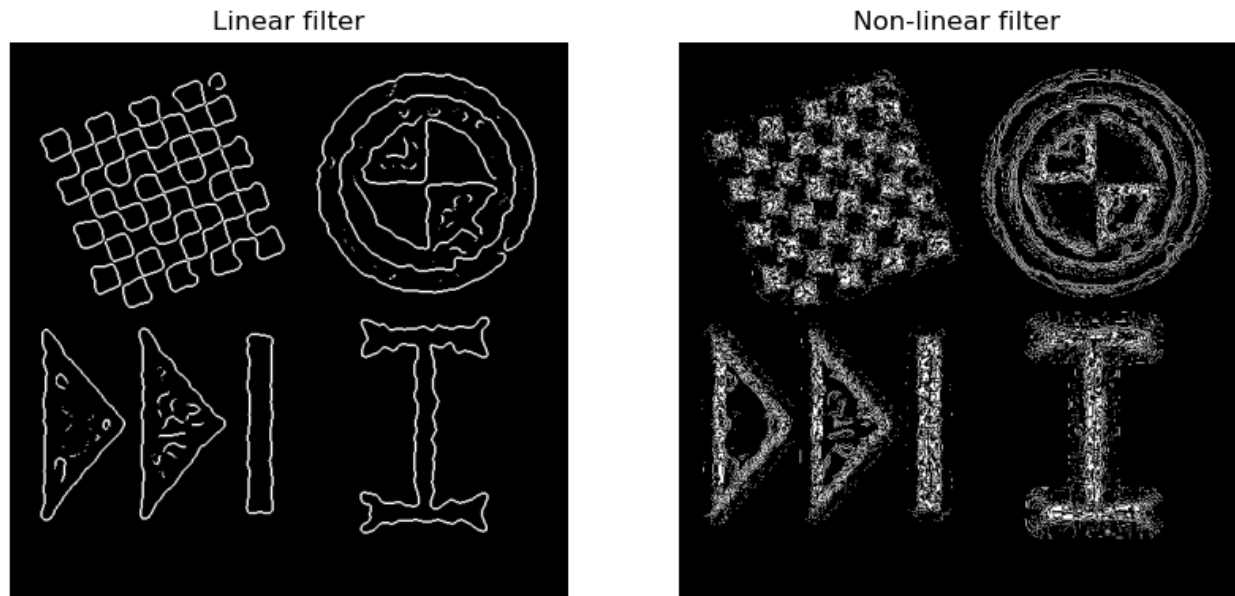
Σχήμα 3:

L1: Η εικόνα μετά το φίλτρο LoG

X1: Η δυαδική εικόμα της L1

Y1: Το περίγραμμα της X1

1.2.4 Απορρίπτονται οι ακμές σε ομαλές περιοχές. Θεωρούμε αυθαίρετα $\theta_{edge} = 0.1$. Το βέλτιστο θ_{edge} θα βρεθεί αργότερα στην εργασία. Εμφανίζονται τα αποτελέσματα.



Σχήμα 4:

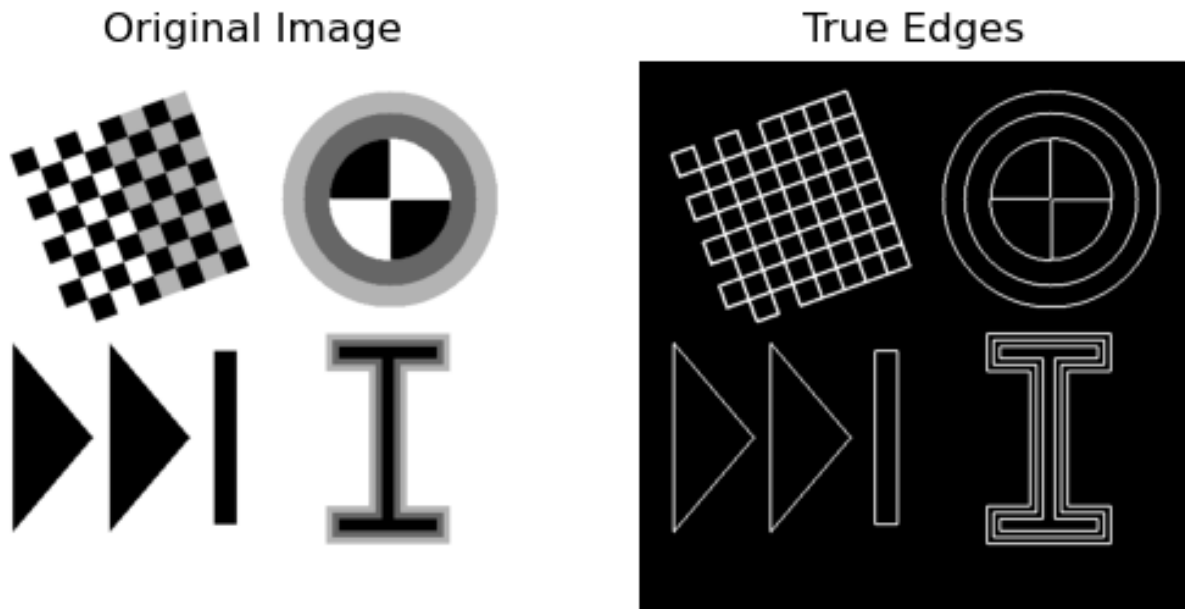
Αριστερά: Το τελικό περίγραμμα της εικόνας με χρήση γραμμικού φίλτρου

Δεξιά: Το τελικό περίγραμμα της εικόνας με χρήση μη γραμμικού μορφολογικού φίλτρου

Τέλος, συνθέτουμε τα παραπάνω βήματα στη ζητούμενη συνάρτηση `EdgeDetect()`

1.3 Αξιολόγηση Αποτελεσμάτων Ανίχνευσης Ακμών

1.3.1 Εφαρμόζουμε στην εικόνα τους κατάλληλους μορφολογικούς τελεστές και κατωφλιοποιούμε το αποτέλεσμα, κατασκευάζουμε την εικόνα T , με τις πραγματικές ακμές. Θεωρώντας $\theta_{ealedge1}$, παίρνουμε το προσδοκώμενο αποτέλεσμα.

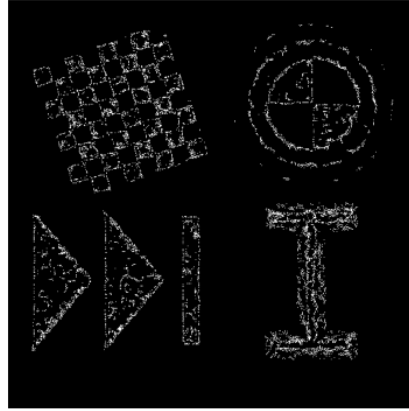
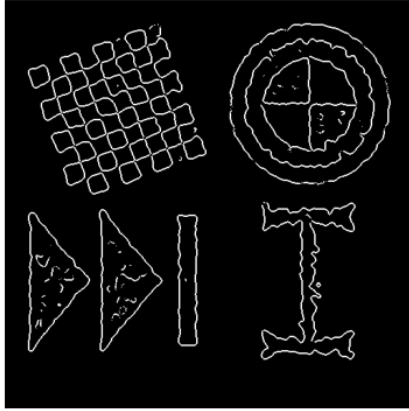


Σχήμα 5: Η αρχική εικόνα και οι ακμές της

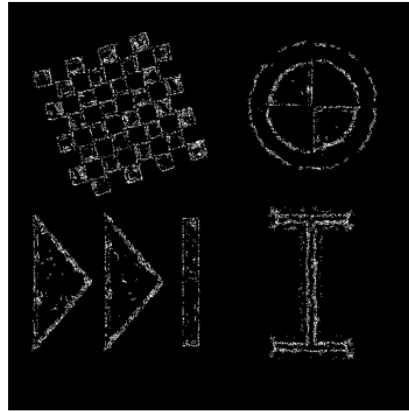
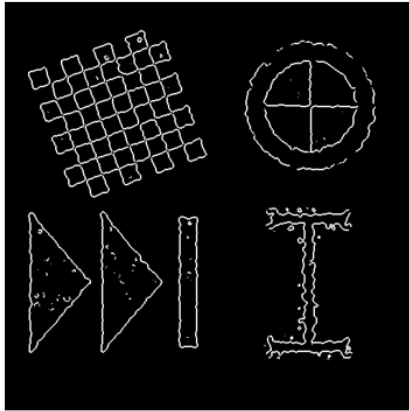
1.3.2 Ορίζεται η συνάρτηση `score()`, που δέχεται μια εικόνα με τις εκτιμώμενες ακμές και την εικόνα με τις πραγματικές ακμές και υπολογίζει την παράμετρο C , σύμφωνα με τον δωσμένο τύπο.

1.3.3 Εφαρμόζουμε την συνάρτηση `EdgeDetect()` στην εικόνα με PSNR=20dB και PSNR=10dB αντίστοιχα, δοκιμάζοντας χειροκίνητα διαφορετικές παραμέτρους, με μέθοδο όμοια του αλγορίθμου Gradient Descent. Οι τιμές που οδήγησαν στα (τοπικά) βέλτιστα `score` φαίνονται στην εικόνα παρακάτω.

PSRN=20dB, $\sigma=4.5$, $\theta=0.16$, Linear (C=0.5435) PSRN=20dB, $\sigma=3$, $\theta=0.2$, Non-Linear (C=0.2663)



PSRN=10dB, $\sigma=3.2$, $\theta=0.2$, Linear (C=0.6145) PSRN=10dB, $\sigma=3.1$, $\theta=0.18$, Non-Linear (C=0.2727)



Σχήμα 6: Τα τελικά αποτελέσματα των δύο αλγορίθμων (L1&L2), μετά από δοκιμές πολλών τιμών παραμέτρων

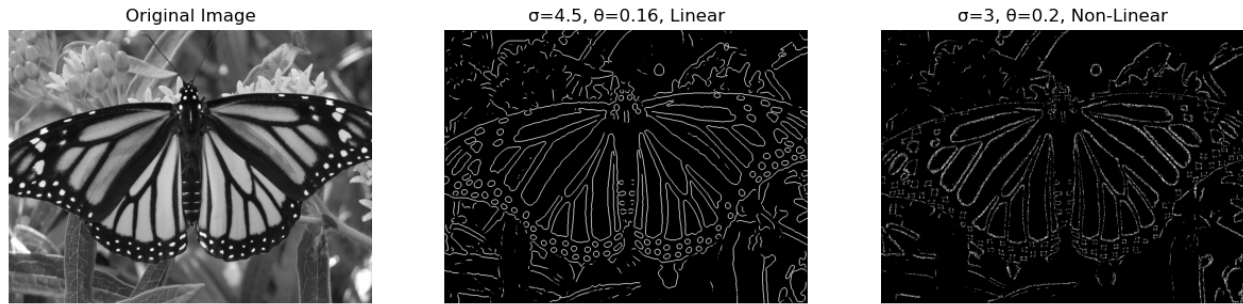
Παρατηρήσεις:

- Οι "βέλτιστες" αυτές τιμές βρέθηκαν προσεγγιστικά χρησιμοποιώντας χειροκίνητη μέθοδο. Για να βρεθούν οι πραγματικά βέλτιστες παράμετροι θα πρέπει να εφαρμοστεί επαναληπτικά ο αλγόριθμος Gradient Descent, από διαφορετικά σημεία εκκίνησης κάθε φορά, για να αποφευχθούν τυχόν τοπικά βέλτιστες λύσεις.
- Ο αλγόριθμος που αξιοποιεί το γραμμικό φίλτρο LoG έχει οδηγήσει σε σημαντικά καλύτερα αποτελέσματα (τουλάχιστον με το εύρος παραμέτρων που δοκιμάστηκαν)
- Το γραμμικό φίλτρο LoG φαίνεται να προσεγγίζει τις ακμές με συνεχείς γραμμές. Αντιθέτως, το μορφολογικό φίλτρο φαίνεται να κατασκευάζει τις ακμές με ανεξάρτητα μεταξύ τους σημεία.
- Επαναλαμβάνοντας το πρόγραμμα με ακριβώς τις ίδιες παραμέτρους οδηγούμαστε κάθε φορά σε ελαφρώς διαφορετικά αποτελέσματα. Θεωρώ ότι αυτό οφείλεται στο στάδιο της θορυβοποίησης που περιλαμβάνει, έστω σε μικρή κλίμακα, τον παράγοντα της "τύχης".

1.4 Εφαρμογή των Αλγορίθμων Ανίχνευσης Ακμών σε Πραγματικές Εικόνες

1.4.1 Φορτώνουμε την εικόνα 'butterfly.jpg' και εφαρμόζουμε τη συνάρτηση EdgeDetect().

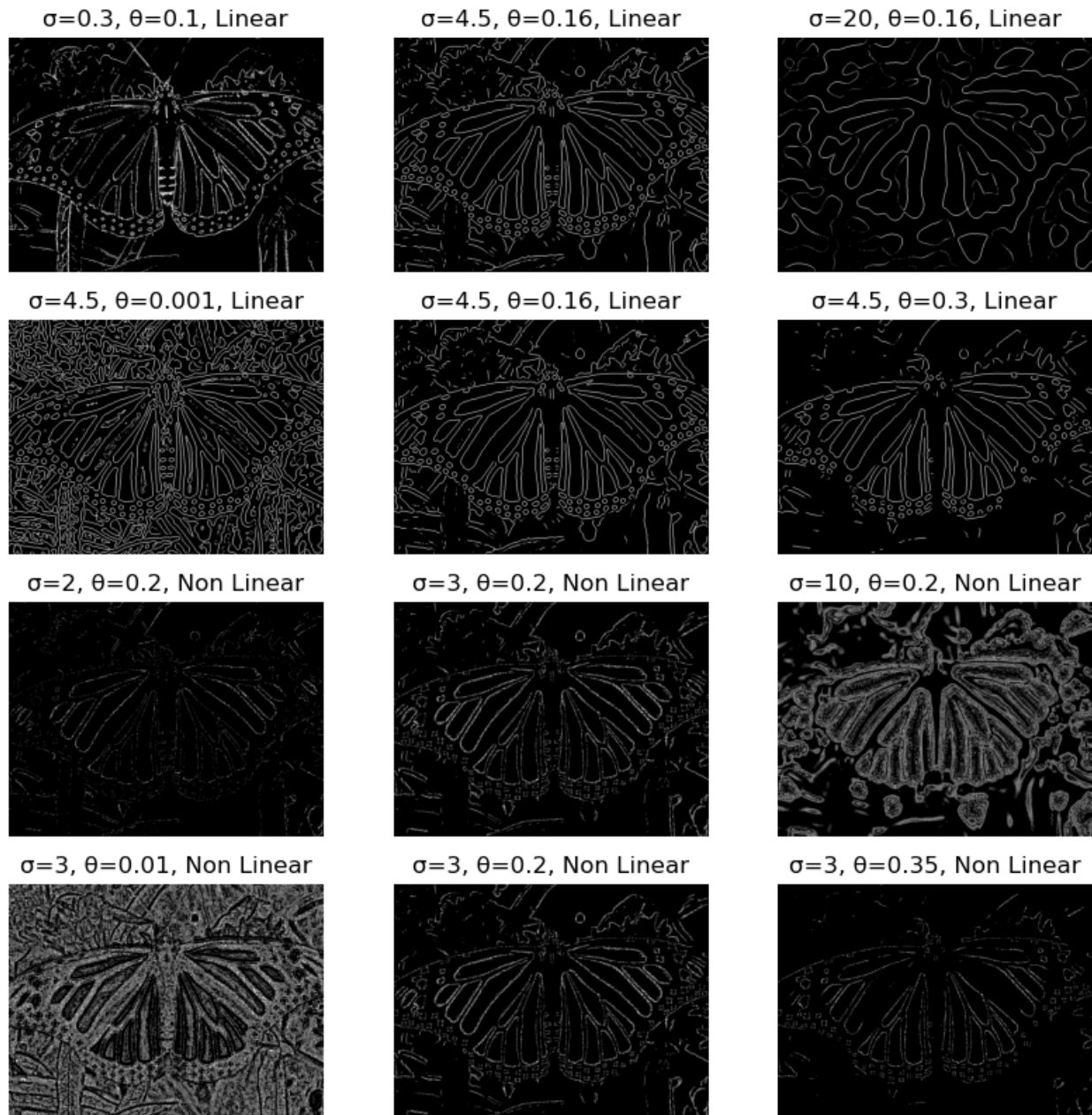
1.4.2 Δοκιμάζουμε όπως προηγουμένως διαφορετικές τιμές για τις παραμέτρους του αλγορίθμου. Οι βέλτιστες που βρεθήκαν φαίνονται στην εικόνα



Σχήμα 7: Τα τελικά αποτελέσματα των δύο αλγορίθμων (L1&L2), μετά από δοκιμές πολλών τιμών παραμέτρων

Παρατηρήσεις:

- Όταν έχουμε μικρό σ_n , τόσο περισσότερες λεπτομέρειες περιλαμβάνονται στις ακμές. στην περίπτωση της πραγματικής εικόνας, το αποτέλεσμα είναι αρκετά ικανοποιητικό,. Αν προσθέσουμε όμως θόρυβο, τότε αυτός λαμβάνεται ως ακμή, πράγμα ανεπιθύμητο.
- Όσο αυξάνουμε το σ_n , συμπεριλαμβάνονται όλο και λιγότερες λεπτομέρειες και οι ακμές που βρίσκουμε ομαλοποιούνται.
- Όμοια με το σ_n , έτσι και το θ_{edge} όταν είναι πολύ μικρό συμπεριλαμβάνονται πολλές "λεπτομέρειες". Εδώ όμως οι "λεπτομέρειες" είναι απλά περιοχές με ελαφρώς απότομη μεταβολή στην φωτεινότητα και που δε θα θεωρούσαμε ακμές. Το τελικό αποτέλεσμα είναι περιοχές με ένα "θορυβώδες" πλήθος ακμών. Σε κάθε περίπτωση λοιπόν το υπερβολικά μικρό θ_{edge} είναι ανεπιθύμητο.
- Καθώς μεγαλώνουμε το θ_{edge} , ο αριθμός αυτών των περιοχών με ασήμαντη μεταβολή στη φωτεινότητα μειώνεται, φτάνοντας ένα βέλτιστο σημείο. Αν συνεχίσουμε να αυξάνουμε περαιτέρω, θα αρχίσουν να περιορίζονται ακόμα και οι επιθυμητές ακμές, μέχρι να φτάσουμε σε μια τελείως σκοτεινή εικόνα.



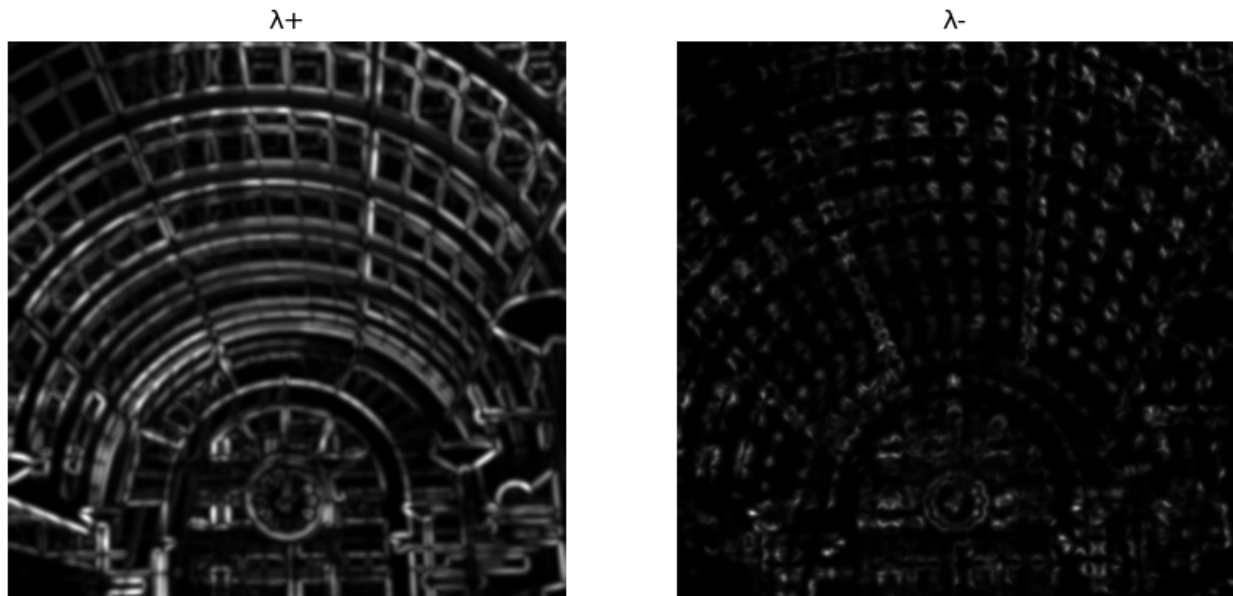
Σχήμα 8: Η εικόνα butterfly.jpg, μετά από δοκιμές διαφόρων παραμέτρων

Μέρος 2: Ανίχνευση Σημείων Ενδιαφέροντος

2.1 Ανίχνευση γωνιών

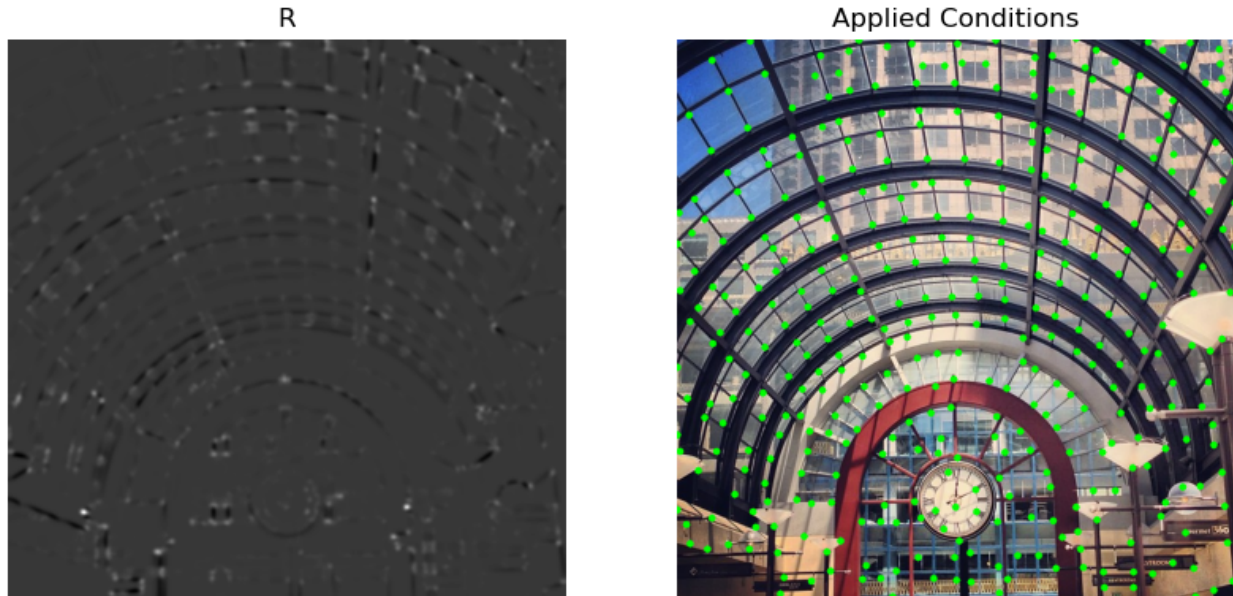
2.1.1 Υπολογίζονται τα στοιχεία του δομικού τελεστή J .

2.1.2 Υπολογίζουμε τις ιδιοτιμές λ_+ , λ_- , βγάζοντας αναμενόμενα αποτελέσματα



Σχήμα 9: λ_+ και λ_-

2.1.3 Υπολογίζουμε το κριτήριο γωνιότητας και εφαρμόζουμε τις δύο συνθήκες. Τέλος, εμφανίζουμε την εικόνα με πράσινες τελείες στα σημεία που απέμειναν.



Σχήμα 10: Μονοκλιμακωτή εύρεση γωνιών

2.2 Πολυκλιμακωτή Ανίχνευση γωνιών

2.2.1 Αφού υλοποιήσουμε μια συνάρτηση που περιλαμβάνει τον αλγόριθμο του μέρους 2.1, τρέχουμε τον αλγόριθμο για πολλές κλίμακες.

2.2.2 Υπολογίζουμε την κανονικοποιημένη LoG για καθεμία από τις κλίμακες. Για κάθε σημείο (x, y) που έχουμε βρει, ελέγχουμε αν $LoG(x, y, \sigma_i) > LoG(x, y, \sigma_{i+1})$ και $LoG(x, y, \sigma_i) > LoG(x, y, \sigma_{i-1})$. Αν δεν ισχύει η συνθήκη, το απορρίπτουμε. Τέλος, οπτικοποιούμε το αποτέλεσμα χρησιμοποιώντας τη συνάρτηση `interest_points_visualization()`.



Σχήμα 11: Πολυκλιμακωτό corner detection

2.3 Ανίχνευση Blobs

2.3.1 Υπολογίζονται οι μερικές παράγωγοι και το κριτήριο R

2.3.2 Χρησιμοποιώντας τις ίδιες συνθήκες με το μέρος 2.1, καταλήγουμε σε ορισμένα blobs, τα οποία σηματοδοτούμε με πράσινες τελείες.



Σχήμα 12: Μονοκλιμακωτό blob detection

2.4 Πολυκλιμακωτή Ανίχνευση Blobs

2.4.1 Όπως και προηγουμένως, υλοποιούμε τη συνάρτηση *BlobDetect()*. Ακολουθώντας όμοια διαδικασία, υλοποιούμε το normalized *LoG* και κρατάμε μόνο τα σημεία που είναι μέγιστα σε σχέση με τις γειτονικές τους κλίμακες. Οπτικοποιούμε το αποτέλεσμα.



Σχήμα 13: Πολυκλιμακωτό blob detection

2.5 Επιτάχυνση με την χρήση Box Filters και Ολοκληρωτικών Εικόνων (Integral Images)

2.5.1 Χρησιμοποιώντας τη συνάρτηση *np.cumsum()* κατά τους άξονες x και y της αρχικής εικόνας (σε grayscale), υπολογίζουμε την αντίστοιχη πολυκλιμακωτή εικόνα.



Σχήμα 14: Η ολοκληρωτική εικόνα της εικόνας εισόδου

2.5.2 Υλοποιούμε τη βοηθητική συνάρτηση $int_sum()$, που δέχεται σαν ορίσματα τις διαστάσεις ενός παραθύρου, μια ολοκληρωτική εικόνα και αρχικές συντεταγμένες (x,y) και υπολογίζει το άθροισμα των σημείων της αρχικής εικόνας στο δοσμένο παράθυρο, χρησιμοποιώντας τον τύπο (18) της εκφώνησης. Έστερα, προκειμένου να υπολογίσουμε τα D_{xx} , D_{yy} , και D_{xy} , διατρέχουμε κάθε σημείο της εικόνας και καλούμε τη συνάρτηση $int_sum()$ (με αρχικό το διατρεχόμενο σημείο), υπολογίζοντας τα αθροίσματα των κατάλληλων περιοχών, ανάλογα με τη συνάρτηση που προσεγγίζουμε. Για το κάθε σημείο, προσθέτουμε ή αφαιρούμε τις περιοχές κατάλληλα, σύμφωνα με τους πυρήνες του σχήματος 2 της εκφώνησης.

2.5.3 Υπολογίζουμε το αντίστοιχο κριτήριο blob.

2.5.4 Για άλλη μια φορά, συνδιάζουμε τα προηγούμενα βήματα σε μια συνάρτηση ($FastBlobDetect()$), η οποία είναι ίδια με την $BlobDetect()$, μόνο που αντί για L_{xy} χρησιμοποιεί τις προσεγγίσεις D_{xy} , καθώς και χρησιμοποιεί το αντίστοιχο προσσεγιστικό R . Χρησιμοποιούμε τη συνάρτηση και εφαρμόζουμε ακριβώς την ίδια διαδικασία με το μέρος 2.4.1.



Σχήμα 15: Πολυκλιμακωτό blob detection με χρήση box filters

Μέρος 3: Εφαρμογές σε Ταίριασμα και Κατηγοριοποίηση Εικόνων με Χρήση Τοπικών Περιγραφητών στα Σημεία Ενδιαφέροντος

3.1. Ταίριασμα Εικόνων υπό Περιστροφή και Αλλαγή Κλίμακας

3.1.1 Αρχικά, οι αλγόριθμοι που αναπτύχθηκαν στα προηγούμενα μέρη οργανώνονται σε μια σειρά από συναρτήσεις και τροποποιούνται ελαφρώς ώστε τα αποτελέσματα να είναι συμβατά με τις συναρτήσεις του `cv24_lab1_part3_utils`. Ύστερα, χρησιμοποιούμε τη συνάρτηση `matching_evaluation()` με inputs τον Harris Detector και τον περιγραφητή SURF.

Listing 1: Αποτελέσματα `matching_evaluation()` για Harris Detector και περιγραφητή SURF

```
1 Avg. Scale Error for Image 1: 0.002
2 Avg. Theta Error for Image 1: 0.109
3
```

3.1.2 Καλούμε τη συνάρτηση `matching_evaluation()` για τις διάφορες εισόδους. Παρατηρούμε ότι την καλύτερη επίδοση παρουσιάζουν οι ανιχνευτές Harris και Hessian, με τον περιγραφητή SURF.

Listing 2: Αποτελέσματα `matching_evaluation()` για διάφορες εισόδους

```
1 1. MONOSCALED HARRIS - SURF DESCRIPTOR
2 Avg. Scale Error for Image 1: 0.003
3 Avg. Theta Error for Image 1: 1.967
4
5 2. MONOSCALED HARRIS - HOG DESCRIPTOR
6 Avg. Scale Error for Image 1: 0.186
7 Avg. Theta Error for Image 1: 22.619
8
9 3. MULTISCALED HARRIS - SURF DESCRIPTOR
10 Avg. Scale Error for Image 1: 0.002
11 Avg. Theta Error for Image 1: 0.109
12
13 4. MULTISCALED HARRIS - HOG DESCRIPTOR
14 Avg. Scale Error for Image 1: 0.097
15 Avg. Theta Error for Image 1: 13.003
16
17 5. MONOSCALED HESSIAN - SURF DESCRIPTOR
18 Avg. Scale Error for Image 1: 0.027
19 Avg. Theta Error for Image 1: 7.759
20
21 6. MONOSCALED HESSIAN - HOG DESCRIPTOR
22 Avg. Scale Error for Image 1: 0.186
23 Avg. Theta Error for Image 1: 7.200
24
25 7. MULTISCALED HESSIAN - SURF DESCRIPTOR
26 Avg. Scale Error for Image 1: 0.001
27 Avg. Theta Error for Image 1: 0.078
28
29 8. MULTISCALED HESSIAN - HOG DESCRIPTOR
30 Avg. Scale Error for Image 1: 0.073
31 Avg. Theta Error for Image 1: 10.553
32
33 9. MULTISCALED SURF - SURF DESCRIPTOR
34 Avg. Scale Error for Image 1: 0.382
35 Avg. Theta Error for Image 1: 29.999
36
37 10. MULTISCALED SURF - HOG DESCRIPTOR
38 Avg. Scale Error for Image 1: 0.445
39 Avg. Theta Error for Image 1: 24.136
40
```

3.2. Κατηγοριοποίηση Εικόνων

3.2.1 Πραγματοποιείται feature detection στα δεδομένα χρησιμοποιώντας Harris Detector και SURF Descriptor. Η διαδικασία διαρκεί 253.512 μονάδες χρόνου.

3.2.2 - 3.2.4 Ακολουθείται η διαδικασία διαχωρισμού δεδομένων, δημιουργίας Bag of Words και κατηγοριοποίησης όπως περιγράφεται στην εκφώνηση και στο example_classification.py. Δυστυχώς, κατά τον διαχωρισμό των δεδομένων προκύπτει ένα IndexError. Έγιναν πολλές δοκιμές τροποποίησης της εισόδου, αλλά χωρίς την πρόσβαση στον πηγαίο κώδικα της cv24_lab1_part3_utils η εύρεση της αιτίας του error ήταν πολύ δύσκολη. Παραθέτω τον κώδικα για αυτό το μέρος, όπως και το error message.

Listing 3: Υλοποίηση βημάτων 3.2.2 - 3.2.4

```
1 accuracies = []
2 for k in range(5):
3     # Split into training set and test set
4     data_train, label_train, data_test, label_test = utils3.createTrainTest(features, k)
5
6     # Perform Kmeans
7     BOF_tr, BOF_ts = utils3.BagOfWords(data_train, data_test)
8
9     # Train an svm on the training set and make predictions on the test set
10    acc, preds, probas = utils3.svm(BOF_tr, label_train, BOF_ts, label_test)
11    accuracies.append(acc)
12
13    print('Mean accuracy for Harris-Laplace with SURF descriptors: {:.3f}%'.format(100.0*np.mean(
14        accuracies)))
```

Listing 4: Αντίστοιχο error message

```
1 -----
2 IndexError                                Traceback (most recent call last)
3 /tmp/ipykernel_9593/482120508.py in <module>
4 2 for k in range(5):
5 3     # Split into training set and test set
6 ----> 4     data_train, label_train, data_test, label_test = utils3.createTrainTest(features,
7     k)
8 5
9 6     # Perform Kmeans
10
11 ~./Documents/School/8th Semester/Computer Vision/Lab/Lab1/cv24_lab1_part3_utils.pyc in
12 createTrainTest(000000000000000000, k)
13
14 ~./Documents/School/8th Semester/Computer Vision/Lab/Lab1/cv24_lab1_part3_utils.pyc in <
15 listcomp>(.0)
16
17 IndexError: list index out of range
```